

C++ Runtime Polymorphism, Upcasting, and Virtual Mechanism

1. Upcasting

Definition:

Upcasting refers to assigning the address of a **derived class object** to a **base class pointer** or **reference**.

Rules and Conditions

1. **Inheritance mode must be public.**
 - In **private** or **protected** inheritance, upcasting is not allowed outside the class hierarchy.
2. Upcasting can be done using either:
 - **Pointer**, or
 - **Reference** to base class.
3. Once upcasting occurs, only **base class members** (accessible ones) can be used.
4. For **non-virtual functions**, the **function of the class whose pointer/reference type** is used gets called.
5. For **virtual functions**, the **function of the class whose object** is pointed to gets called.

Example

2. Binding (Function Resolution)

Definition:

Binding means associating a function call with its actual function body.

Types of Binding

1. **Early Binding (Static Binding)**
 - Default in C++.
 - Happens for **non-virtual** functions.
 - The compiler binds the call at compile time based on **type of pointer or reference**, not the object.
 - Used when the function call can be resolved at compile time.
2. **Late Binding (Dynamic Binding)**
 - Happens only when:
 - The function is declared **virtual** in the base class, **and**
 - It is called through a **base class pointer or reference**.
 - The compiler defers the call until runtime and executes based on **content of pointer (actual object)**.

Summary

Function Type	Pointer Type	Binding	Function Executed
---------------	--------------	---------	-------------------

Non-virtual	Base*	Early	Base function
Virtual	Base*	Late	Derived function

3. Redefinition vs Overriding

Concept	Virtual Keyword	Binding	Rule	Behavior
Redefinition	Absent	Early	Same name in child hides base version	Determined by pointer type
Overriding	Present	Late	Base function is virtual and redefined in child	Determined by object type

Compiler's Action:

When you override a parent class method in the child:

- Even if you make it **private**, overriding is valid.
- The compiler checks:
 1. The **content of the pointer/reference** (actual object type),
 2. Whether the base class version of the function is **virtual and accessible**.
- If so, compiler sets up runtime dispatch using **VTABLE** and **VPTR**.

4. Dynamic Polymorphism – 3 Ways

Runtime polymorphism (late binding) can be achieved in **three ways**:

1. Using Base Class Pointer

```
Base* ptr = new Derived;
ptr->disp();
```

2. Using Base Class Reference

```
Derived d;
Base& ref = d;
ref.disp();
```

3. Using Array of Base Class Pointers

```
Animal* arr[3];
arr[0] = new Dog;
arr[1] = new Cat;
arr[2] = new Tiger;
```

```
for (int i = 0; i < 3; i++) arr[i]->makeSound();
```

Here, each pointer stores address of a derived object, and function calls resolve dynamically.

5. Virtual Function

Definition:

A virtual function is a base class function declared with the keyword **virtual**, allowing **derived classes** to override it and enabling **runtime polymorphism**.

Key Points

1. Virtual functions work **only via base class pointer or reference**.

2. The compiler ensures **late binding** only when the function is virtual.
3. If a class has at least one virtual function, it is automatically **inherited as virtual**.
4. Virtuality is maintained across derived classes unless the function is re-declared as final or non-virtual.
5. Even if the derived class **does not override**, it still receives a **VTABLE** containing the parent's virtual function address.
6. The **address order** of virtual functions in the table depends on their **declaration order** in the class.

6. Virtual Table (VTABLE) and VPTR Mechanism

When a class contains at least one virtual function, the compiler automatically generates a **Virtual Table (VTABLE)** for that class.

Conceptual Layout

Each object of a class with virtual functions has a **hidden data member** called **VPTR (Virtual Pointer)** that points to its class's VTABLE.

```
class Base {
public:
    virtual void disp();
};

class Child : public Base {
public:
    void disp() override;
};
```

Compiler Actions

1. Compiler creates **one VTABLE per class**.
2. Each entry in VTABLE stores the **address of a virtual function**.
3. Each object has a hidden **VPTR**, initialized by constructor.
4. When a virtual function is called:
 - Compiler checks the **content of pointer** (object type).
 - Fetches the **VPTR** from the object.
 - Accesses the **VTABLE** through VPTR.
 - Invokes the corresponding function using the **stored address**.

Visualization

Class	Object Addr	VPTR	VTABLE Entries (Function Addresses)
Base	1000	→ 5000	disp() → 3000
Child	1300	→ 9000	disp() → 4000

If child doesn't override, the compiler fills its VTABLE entry with the **base function address**.

Multiple Virtual Functions Example

```
class Base {
public:
```

```

        virtual void disp1();
        virtual void disp2();
    };
    class Child : public Base {
    public:
        void disp2() override;
        virtual void disp3();
    };

```

VTABLE Layout:

Function Index	Base::VTABLE	Child::VTABLE
0	disp1 → Base::disp1	disp1 → Base::disp1
1	disp2 → Base::disp2	disp2 → Child::disp2
2	—	disp3 → Child::disp3

Order of entries in VTABLE strictly follows the **declaration order** in the base class.

7. Virtual Destructors

When deleting a derived object through a base pointer:

```

Base* b = new Derived;
delete b;

```

If the base destructor is **not virtual**, only the **base class destructor** executes, leading to **memory leaks** if the derived class allocates dynamic memory.

Solution:

Declare **virtual destructor** in the base class.

```

class Base {
public:
    virtual ~Base() { cout << "Base destroyed\n"; }
};
class Derived : public Base {
public:
    ~Derived() { cout << "Derived destroyed\n"; }
};

```

Output:

Derived destroyed

Base destroyed

Compiler Behavior:

At delete b;, compiler checks the **content of pointer**, finds VPTR of Derived, fetches destructor address from VTABLE, and executes destructors in correct order.

8. Virtual Function Calls from Constructors and Destructors

When a virtual function is called **inside a constructor or destructor**, **early binding** occurs.

Reason:

During construction or destruction, the object's **VPTR is still bound to the current class**, not to the most-derived class.

```

class Base {
public:
    Base() { disp(); } // Calls Base::disp()

```

```

        virtual void disp() { cout << "Base disp\n"; }
    };
    class Derived : public Base {
    public:
        void disp() override { cout << "Derived disp\n"; }
    };

```

Output:

Base disp (because Derived part not yet constructed).

Same rule applies in destructors.

9. Virtual Functions Called from Member Functions

If a **member function** calls a virtual function:

- **Called by object:** → Early binding
- **Called by pointer/reference:** → Late binding

```

class Base {
public:
    virtual void disp() { cout << "Base disp\n"; }
    void run() { disp(); } // Depends on caller
};
class Child : public Base {
public:
    void disp() override { cout << "Child disp\n"; }
};
int main() {
    Base b; b.run(); // Early → Base disp
    Child c; c.run(); // Early → Child disp
    Base* p = new Child;
    p->run(); // Late → Child disp
}

```

10. Internal Structure of VTABLE

The VTABLE stores:

1. **Addresses of all virtual functions.**
2. **Address of virtual destructor** (if present).
3. **Information about pure virtual functions.**
4. **RTTI (Run-Time Type Information)** entries for type identification.

Each class gets **one VTABLE**, each object gets **one VPTR per base class containing virtuals**.

11. Notes on Multiple Inheritance

If multiple base classes contain virtual functions:

- Each base class contributes its own **VTABLE** and **VPTR**.
- The derived object stores **multiple VPTRs**, one for each base.

12. Virtual Constructor (Factory Concept)

Constructors cannot be declared virtual because object creation type must be known at compile time. However, the behavior can be mimicked using a **Factory Method**.

```
class Shape {
public:
    virtual void draw() = 0;
    static Shape* createShape(int choice);
};
Shape* Shape::createShape(int choice) {
    switch (choice) {
        case 1: return new Circle;
        case 2: return new Triangle;
        default: return nullptr;
    }
}
```

This pattern is informally known as a **virtual constructor**.

13. Summary Table

Concept	Meaning	Binding Type	Notes
Upcasting	Derived → Base	Depends	Enables polymorphism
Downcasting	Base → Derived	Explicit (dynamic_cast)	May fail if object not derived
Redefinition	Non-virtual same name	Early	Hides base version
Overriding	Virtual same signature	Late	Enables dynamic binding
Virtual Function	Late-bound member	Runtime	Declared with virtual
Virtual Destructor	Destroys derived correctly	Runtime	Always use if base pointer delete
VTABLE	Stores function addresses	Runtime	One per class
VPTR	Pointer to VTABLE	Runtime	One per object
Virtual Constructor	Factory method	Runtime	Creates derived via base pointer
RTTI	Runtime type identification	Runtime	Stored in VTABLE
Object Slicing	Derived → Base object copy	Compile time	Loses derived data
